Practice · Whole programs

Each question lists its options; the right one is marked **Right answer**. *Feedback* is what students see after they answer. In a typed question the blank is in square brackets, with the accepted answers separated by `|`.

(mc) L1 · What this program does

Read the whole program before you answer.

```
#include <stdio.h>
int main(void) {
    double c;
    printf("Celsius: ");
    scanf("%lf", &c);
    printf("Fahrenheit: %.2f\n", c * 9 / 5 + 32);
    return 0;
}
```

What does it do?

- It prints the word `Celsius` and stops.
- It converts Fahrenheit to Celsius.
- **Right answer:** It asks for a temperature in Celsius and prints it in Fahrenheit.
- It prints `c * 9 / 5 + 32` on the screen.

Feedback: Read it top to bottom. `printf("Celsius: ")` asks. `scanf` reads your number into `c`. The last `printf` prints the answer of `c * 9 / 5 + 32` in the `%.2f` hole, with the label `Fahrenheit:`. That is the temperature converter from the practice lab.

(mc) L1 · Which line asks the question

Same program:

```
#include <stdio.h>
int main(void) {
    double c;
    printf("Celsius: ");
    scanf("%lf", &c);
    printf("Fahrenheit: %.2f\n", c * 9 / 5 + 32);
    return 0;
}
```

Which line makes the program stop and wait for you to type?

- Line 3, `double c;`
- Line 4, `printf("Celsius: ");`
- **Right answer:** Line 5, `scanf("%lf", &c);`
- Line 6, the second `printf`

Feedback: `printf` only puts words on the screen and moves on. `scanf` is the line that waits. The screen shows `Celsius:` because of line 4, but the blinking cursor waiting for you is line 5.

(mc) L1 · The shape of every program this week

Every program this week has the same four steps in the same order. Which order?

- Calculate, ask, read, print.
- **Right answer:** Ask, read, calculate, print.
- Read, ask, print, calculate.
- Print, calculate, ask, read.

Feedback: Look at the circle calculator on slide 14: `printf("Radius: ")` asks, `scanf` reads, `area = ...` calculates, `printf("Area: ...")` prints. Ask, read, calculate, print. When you write your own program, write those four steps as comments first, then fill them in.

(mc) L1 · What is missing

This program is meant to read a radius and print the area of a circle.

```
#include <stdio.h>
int main(void) {
    double r, area;
    const double PI = 3.14159;
    printf("Radius: ");
    area = PI * r * r;
    printf("Area: %.2f\n", area);
    return 0;
}
```

It compiles, though `-Wall` warns that `r` is used uninitialized, and the area it prints is `0.00` or nonsense. Which step is missing?

- The calculate step: there is no formula.
- **Right answer:** The read step: there is no `scanf`, so `r` never gets a value.
- The print step: there is no `printf`.
- The `return 0;` at the end.

Feedback: The program asks `Radius: ` and then never reads your answer. `r` is a box that was never filled, so `PI * r * r` uses whatever rubbish was in the box. The missing line is `scanf("%lf", &r);` right after the question.

(mc) L1 · Which include lines

This program prints the distance between two points using `sqrt`.

```
int main(void) {
    double x1, y1, x2, y2, d;
    printf("Four numbers: ");
    scanf("%lf %lf %lf %lf", &x1, &y1, &x2, &y2);
    d = sqrt((x2 - x1) * (x2 - x1) + (y2 - y1) * (y2 - y1));
    printf("Distance: %.2f\n", d);
    return 0;
}
```

The lines at the top are missing. Which lines does it need?

- `#include <stdio.h>` only.
- `#include <math.h>` only.

- **Right answer:** `#include <stdio.h>` and `#include <math.h>`.
- No include lines. They are optional.

Feedback: `printf` and `scanf` come from `stdio.h`. `sqrt` comes from `math.h`. This program uses all three, so it needs both lines. Miss `stdio.h` and the compiler complains about `printf`. Miss `math.h` and it complains about `sqrt`.

(mc) L1 · Which variable holds what

Look at the bill splitter.

```
#include <stdio.h>
int main(void) {
    double total;
    int people;
    printf("Total: ");
    scanf("%lf", &total);
    printf("People: ");
    scanf("%d", &people);
    printf("Each pays: %.2f\n", total / people);
    return 0;
}
```

Why is `total` a `double` but `people` an `int`?

- No reason. Both could be `int`.
- **Right answer:** A bill can have a decimal part, like 100.50. A number of people is always a whole number.
- `scanf` can only read one `double` per program.
- `double` is for the first number and `int` for the second.

Feedback: Choose the type from what the box will hold. Money has decimals, so `double`. You cannot have 2.5 people, so `int`. The holes follow the types: `%lf` for `total`, `%d` for `people`.

(mc) L1 · Where the answer goes

```
#include <stdio.h>
int main(void) {
    int a, b, sum;
    printf("Two numbers: ");
    scanf("%d %d", &a, &b);
    sum = a + b;
    printf("Sum = %d\n", sum);
    return 0;
}
```

What does the line `sum = a + b;` do?

- It prints the sum.
- It checks whether `sum` is equal to `a + b`.
- **Right answer:** It works out `a + b` and stores the answer in the box `sum`.
- It reads two numbers from the keyboard.

Feedback: `=` means "work out the right side, then put it in the box on the left". Nothing is printed. That happens on the next line, where `%d` is filled with the value of `sum`.

(mc) L1 · The same program without the extra box

Here is the same sum program written shorter:

```
#include <stdio.h>
int main(void) {
    int a, b;
    printf("Two numbers: ");
    scanf("%d %d", &a, &b);
    printf("Sum = %d\n", a + b);
    return 0;
}
```

Is this correct?

- **Right answer:** Yes. A calculation can be put straight into a `printf` hole. No `sum` box is needed.
- No. You must always store a result in a variable before printing it.
- No. `a + b` inside `printf` prints the text `a + b`.
- No. `printf` can only take one variable.

Feedback: Both versions work and print the same thing. A hole can be filled by any calculation, not only by a variable. Use a variable when you need the answer more than once, or when it makes the program easier to read.

(mc) L1 · The compile line

You saved the distance program as `distance.c`. It uses `sqrt`. You are on Linux or WSL. Which compile line is right?

- `gcc -Wall distance.c`
- `gcc -Wall distance.c -o distance -lm distance`
- **Right answer:** `gcc -Wall distance.c -o distance -lm`
- `./distance -lm`

Feedback: `-Wall` asks for all warnings, `-o distance` names the output, and `-lm` links the math library, which Linux and WSL need for `sqrt` and `pow`. The `-lm` goes at the end. On Windows with the course installer and on a Mac, `-lm` is not needed but does no harm.

(mc) L2 · Predict: temperature converter

```
#include <stdio.h>
int main(void) {
    double c;
    printf("Celsius: ");
    scanf("%lf", &c);
    printf("Fahrenheit: %.2f\n", c * 9 / 5 + 32);
    return 0;
}
```

You run it and type `100`. What does the screen show after your number?

- `Fahrenheit: 212`
- **Right answer:** `Fahrenheit: 212.00`
- `Fahrenheit: 100.00`
- `Fahrenheit: c * 9 / 5 + 32`

Feedback: `c` is 100. Then `100 * 9` is 900, `900 / 5` is 180, `180 + 32` is 212. The hole is `%.2f`, so it shows two digits after the point: `212.00`. Multiplication and division happen before the addition, left to right.

(mc) L2 · Predict: the same program with int

The converter again, but `c` is now an `int` and the hole is `%d`.

```
#include <stdio.h>
int main(void) {
    int c;
    printf("Celsius: ");
    scanf("%d", &c);
    printf("Fahrenheit: %d\n", c * 9 / 5 + 32);
    return 0;
}
```

You type `37`. What does it show?

- `Fahrenheit: 98.60`
- `Fahrenheit: 98.6`
- **Right answer:** `Fahrenheit: 98`
- `Fahrenheit: 99`

Feedback: Everything is `int`, so every step throws away decimals. `37 * 9` is 333, `333 / 5` is 66 (66.6 with the .6 thrown away), `66 + 32` is 98. The true answer is 98.6. This is why the practice lab says to use `double` for the temperature.

(mc) L2 · Predict: bill splitter

```
#include <stdio.h>
int main(void) {
    double total;
    int people;
    printf("Total: ");
    scanf("%lf", &total);
    printf("People: ");
    scanf("%d", &people);
    printf("Each pays: %.2f\n", total / people);
    return 0;
}
```

You type `50` for the total and `4` for the people. What does it show?

- `Each pays: 12`
- **Right answer:** `Each pays: 12.50`
- `Each pays: 12.5`
- `Each pays: 13.00`

Feedback: `total` is a `double`, so `total / people` keeps the decimals: 50 / 4 is 12.5. The hole `%.2f` shows exactly two digits after the point, so `12.50`. If `total` were an `int`, the answer would be 12.

(mc) L2 · Predict: circle with radius 2

```
#include <stdio.h>
int main(void) {
    double r, area, c;
    const double PI = 3.14159;
    printf("Radius: ");
    scanf("%lf", &r);
    area = PI * r * r;
    c = 2 * PI * r;
    printf("Area: %.2f\n", area);
    printf("Circumference: %.2f\n", c);
    return 0;
}
```

You type `2`. What are the two lines it prints?

- `Area: 6.28` and `Circumference: 12.57`
- **Right answer:** `Area: 12.57` and `Circumference: 12.57`
- `Area: 12.57` and `Circumference: 6.28`
- `Area: 12.566360` and `Circumference: 12.566360`

Feedback: With radius 2 the two answers happen to be the same: area is 3.14159 × 2 × 2 = 12.566, and circumference is 2 × 3.14159 × 2 = 12.566. `%.2f` rounds both to `12.57`. Try radius 1 or 3 and they differ.

(mc) L2 · Find the bug: no output for the second number

```
#include <stdio.h>
int main(void) {
    int a, b;
    printf("Two numbers: ");
    scanf("%d", &a, &b);
    printf("%d and %d\n", a, b);
    return 0;
}
```

You type `4 9` and the screen shows `4 and 0`, or `4 and` followed by a nonsense number. `-Wall` says `too many arguments for format`. Which line is wrong?

- Line 3, `int a, b;`
- **Right answer:** Line 5, `scanf("%d", &a, &b);` has one hole for two variables.
- Line 6, `printf("%d and %d\n", a, b);`
- Line 4, the question has no `\n`.

Feedback: One hole, one value read. `scanf("%d", &a, &b)` reads only `a`. The 9 is never read and `b` stays empty, so a rubbish number is printed. The line needs two holes: `scanf("%d %d", &a, &b);`. The number of holes must match the number of variables in `scanf` just as in `printf`.

(mc) L2 · Find the bug: it prints 0.00

```
#include <stdio.h>
int main(void) {
    double price;
    printf("Price: ");
    scanf("%f", &price);
    printf("Price: %.2f\n", price);
    return 0;
}
```

You type `12.5` and it prints `Price: 0.00`. What is wrong?

- `%.2f` in the `printf` should be `%lf`.
- **Right answer:** `%f` in the `scanf` should be `%lf`, because `price` is a `double`.
- `price` should be an `int`.
- The `&` before `price` should be removed.

Feedback: For `scanf`, a `double` needs `%lf`. `%f` in `scanf` means "a `float`", a smaller type, so the value lands in the box wrongly and you see 0.00 or rubbish. `-Wall` warns you about exactly this. `printf` is different: there `%f` and `%.2f` are right for a `double`.

(mc) L2 · Error or warning

The two bugs in this program are the ones from slide 15.

```
#include <stdio.h>
int main(void) {
    int a, b;
    printf("Two numbers: ");
    scanf("%d %d", a, b);
    printf("Sum = %d\n", a + b)
    return 0;
}
```

You compile with `gcc -Wall`. What happens?

- Two errors are printed and no program is made.
- Two warnings are printed and a working program is made.
- **Right answer:** Warnings for the missing `&` and one error for the missing `;`. No program is made until the error is fixed.
- The compile is silent and the program runs.

Feedback: The missing semicolon is an error: the compiler cannot understand the program and makes nothing. The missing `&` is a warning: the compiler can make a program, but it will run wrong. `gcc -Wall` prints both kinds at once. Fix the error first, compile again, then deal with the warnings.

(mc) L2 · Predict: distance

```
#include <stdio.h>
#include <math.h>
int main(void) {
    double x1, y1, x2, y2, d;
    printf("Four numbers: ");
    scanf("%lf %lf %lf %lf", &x1, &y1, &x2, &y2);
```

```
    d = sqrt((x2 - x1) * (x2 - x1) + (y2 - y1) * (y2 - y1));
    printf("Distance: %.2f\n", d);
    return 0;
}
```

You type `1 1 4 5`. What does it show?

- `Distance: 3.00`
- `Distance: 4.00`
- **Right answer:** `Distance: 5.00`
- `Distance: 25.00`

Feedback: `x2 - x1` is 3 and `y2 - y1` is 4. Then 3 × 3 + 4 × 4 is 9 + 16 = 25. The square root of 25 is 5, printed with two decimals as `5.00`. The 3-4-5 triangle again.

(mc) L2 · Why is the answer 2.00 and not 2.50

```
#include <stdio.h>
int main(void) {
    int total, people;
    printf("Total and people: ");
    scanf("%d %d", &total, &people);
    printf("Each pays: %.2f\n", total / people);
    return 0;
}
```

You type `10 4`. Instead of `Each pays: 2.50` the screen shows `Each pays: 0.00` or a nonsense number, and `-Wall` warns about the last line. What is the cause?

- `10 / 4` is 2.5, and `%.2f` cannot show 2.5.
- **Right answer:** `total / people` is `int` divided by `int`, so it is the `int` 2, and `%.2f` is the wrong hole for an `int`.
- The `scanf` should have one hole.
- `people` should come before `total`.

Feedback: Two `int` boxes divided give an `int`, so the .5 is thrown away before printing. Then `%.2f` expects a `double` and gets an `int`, so the output is rubbish and `-Wall` warns. Two fixes: make `total` a `double` and read it with `%lf`, or write `total / 2.0`. The type of the answer follows the types of the boxes, not the hole.

(mc) L2 · Predict: two printf lines and no newline

```
#include <stdio.h>
int main(void) {
    int n = 5;
    printf("n is %d", n);
    printf("%d squared is %d\n", n, n * n);
    return 0;
}
```

What does it show?

- `n is 5` on one line and `5 squared is 25` on the next.

- **Right answer:** `n is 55 squared is 25` on one line.
- `n is 5 5 squared is 25` on one line.
- `n is %d5 squared is 25`

Feedback: The first `printf` has no `\n`, so the second one continues on the same line, right after the 5, with no space. That gives `n is 5` followed straight away by `5 squared is 25`. When your output runs together, look for a missing `\n`.

(fib) L3 · Complete the converter: the read line

Complete the missing piece so the program reads the temperature.

```
#include <stdio.h>
int main(void) {
    double c;
    printf("Celsius: ");
    scanf("[%lf]", &c);
    printf("Fahrenheit: %.2f\n", c * 9 / 5 + 32);
    return 0;
}
```

Feedback: `%lf`. `c` is a `double`, and `scanf` reads a `double` with `%lf`.

(fib) L3 · Complete the converter: the formula

Complete the formula. Fahrenheit is Celsius times 9, divided by 5, plus 32. Type it with single spaces, exactly like `c * 9 / 5 + 32`.

```
#include <stdio.h>
int main(void) {
    double c;
    printf("Celsius: ");
    scanf("%lf", &c);
    printf("Fahrenheit: %.2f\n", [c * 9 / 5 + 32|c*9/5+32|c * 9.0 / 5 + 32|c*9.0/5+32]);
    return 0;
}
```

Feedback: `c * 9 / 5 + 32`. Multiply first, then divide, then add. Because `c` is a `double`, the division keeps its decimals.

(fib) L3 · Complete the splitter: the two read lines

Complete the two holes. `total` is a `double` and `people` is an `int`.

```
#include <stdio.h>
int main(void) {
    double total;
    int people;
    printf("Total: ");
    scanf("[%lf]", &total);
    printf("People: ");
    scanf("[%d]", &people);
    printf("Each pays: %.2f\n", total / people);
    return 0;
```

```
}
```

Feedback: `%lf` for the `double`, `%d` for the `int`. The hole follows the type of the box.

(fib) L3 · Complete the splitter: the print line

Complete the hole so the amount is printed with two digits after the point.

```
#include <stdio.h>
int main(void) {
    double total;
    int people;
    printf("Total: ");
    scanf("%lf", &total);
    printf("People: ");
    scanf("%d", &people);
    printf("Each pays: [%.2f]\n", total / people);
    return 0;
}
```

Feedback: `%.2f`. Money always gets two digits after the point.

(fib) L3 · Complete the circle: the area line

Complete the calculation of the area. Type it with single spaces, like `PI * r * r`.

```
#include <stdio.h>
int main(void) {
    double r, area;
    const double PI = 3.14159;
    printf("Radius: ");
    scanf("%lf", &r);
    area = [PI * r * r|PI*r*r|r * r * PI|r*r*PI];
    printf("Area: %.2f\n", area);
    return 0;
}
```

Feedback: `PI * r * r`. Area is π times the radius squared, and in C you write the star each time.

(fib) L3 · Complete the circle: the diameter

The extension on slide 14 asks for the diameter too. Complete the missing calculation. Type it like `2 * r`.

```
#include <stdio.h>
int main(void) {
    double r, d;
    printf("Radius: ");
    scanf("%lf", &r);
    d = [2 * r|2*r|r * 2|r*2];
    printf("Diameter: %.2f\n", d);
    return 0;
}
```

Feedback: `2 * r`. The diameter is twice the radius. `2r` is not C; the star must be there.

(fib) L3 · Complete the distance: the include line

Complete the second include line so `sqrt` exists.

```
#include <stdio.h>
#include [<math.h>|math.h]
int main(void) {
    double x1, y1, x2, y2, d;
    printf("Four numbers: ");
    scanf("%lf %lf %lf %lf", &x1, &y1, &x2, &y2);
    d = sqrt((x2 - x1) * (x2 - x1) + (y2 - y1) * (y2 - y1));
    printf("Distance: %.2f\n", d);
    return 0;
}
```

Feedback: `<math.h>`. The angle brackets and the `.h` are part of it. This line gives you `sqrt`, `pow`, `round`, `ceil` and `floor`.

(fib) L3 · Complete the distance: the read line

Complete the four variables of the `scanf`. Type them with a comma and one space between, like `&x1, &y1, &x2, &y2`.

```
#include <stdio.h>
#include <math.h>
int main(void) {
    double x1, y1, x2, y2, d;
    printf("Four numbers: ");
    scanf("%lf %lf %lf %lf", [&x1, &y1, &x2, &y2|&x1,&y1,&x2,&y2]);
    d = sqrt((x2 - x1) * (x2 - x1) + (y2 - y1) * (y2 - y1));
    printf("Distance: %.2f\n", d);
    return 0;
}
```

Feedback: `&x1, &y1, &x2, &y2`. Four holes, four variables, each with its own `&`, in the same order as the holes.

(fib) L3 · Fix slide 15: the scanf line

This is the program from slide 15 with the semicolon already fixed. Complete the `scanf` line so it works. Type it like `&a, &b`.

```
#include <stdio.h>
int main(void) {
    int a, b;
    printf("Two numbers: ");
    scanf("%d %d", [&a, &b|&a,&b]);
    printf("Sum = %d\n", a + b);
    return 0;
}
```

Feedback: `&a, &b`. Without the `&`, the program compiles with a warning and then crashes or prints a wrong sum. Type `4 9` and the fixed program prints `Sum = 13`.